

MiniScheme Comparative Implementation

CMPS 3500 — Programming Languages | Final Project | Group 06

Team Members / Main Roles

- Chazz — Common Lisp functional evaluator, run_all.sh support, testing
- Emilio — Procedural C++ implementation
- Stephanie — Object-Oriented Java implementation
- Isaac — Documentation, testing, and demo support

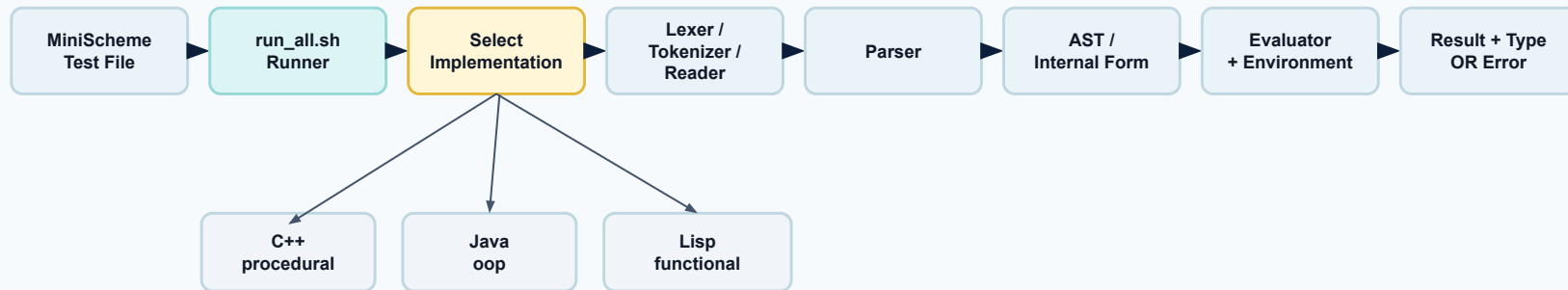
Project Goal

Build the same MiniScheme expression engine in three paradigms: procedural C++, object-oriented Java, and functional Common Lisp.

Each version reads MiniScheme test files, evaluates expressions, and reports either a result/type or a standardized error through run_all.sh.

Project Architecture

All three implementations follow the same basic interpreter pipeline.



What each stage means

- Lexer / Tokenize / Reader: breaks raw input into pieces
- Parser: checks syntax and builds an internal expression form
- AST / Internal Form: how code is represented before evaluation
- Evaluator: applies MiniScheme rules using the environment
- Output: result + type, or a standardized error

Overview

The implementations are different, but the overall pipeline is the same. A test file goes through `run_all.sh`, gets sent to one implementation, gets parsed into an internal form, and then the evaluator returns either a value with a type or a clean error.

Procedural Implementation — C++

Focus: explicit steps, data structures, and direct control flow.

Language / Folder

C++ / procedural
Builds with g++ and runs through
run_all.sh.

How input is represented

The C++ version reads the whole file, splits the input into tokens, and represents expressions using a Node-style tree structure.

How evaluation works

Each Node is evaluated step by step. The environment structure stores variables and functions so let, lambda, define, recursion, and cond can work.

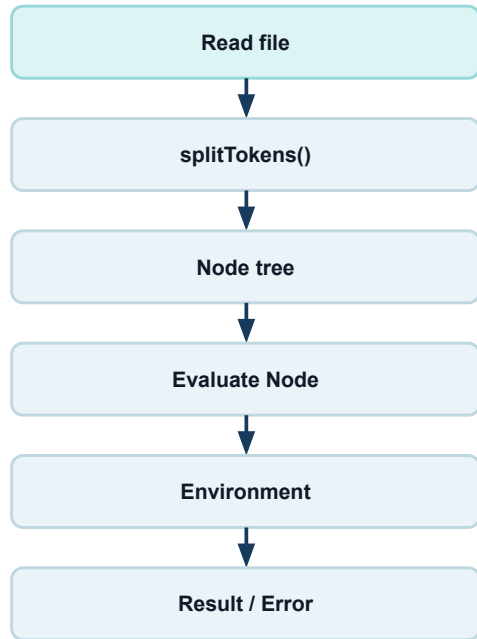
Main strength

The main strength is that the control flow is direct. It is easier to point to the order of operations: read, tokenize, parse, evaluate, print output.

Main challenge

The tradeoff is that more pieces have to be managed manually, including node structure, environment handling, and error output.

C++ flow



Object-Oriented Implementation — Java

Focus: classes, expression objects, and separated responsibilities.

Language / Folder

Java / oop
Builds with javac and runs through
run_all.sh.

How input is represented

The Java version uses a Tokenizer and Parser to turn input into expression objects, such as Num, Bool, If, Let, Lambda, Cond, and function calls.

How evaluation works

Each expression type has its own evaluation behavior. Environment stores bindings, and Closure stores a lambda's body plus the environment where it was created.

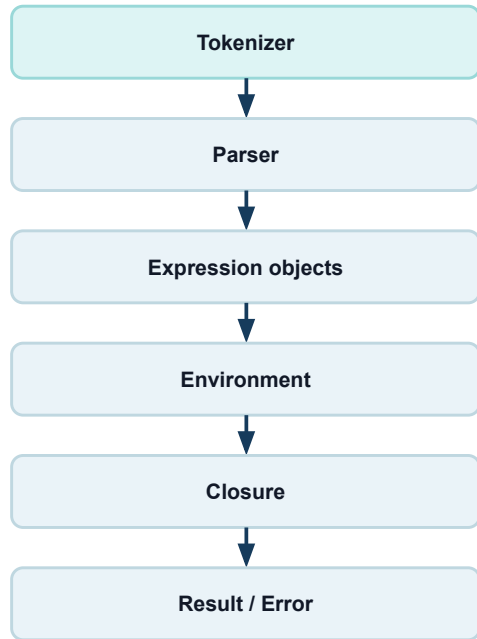
Main strength

The main strength is organization. Classes separate responsibilities, so adding or explaining one expression type is cleaner.

Main challenge

The tradeoff is more boilerplate. Java made the structure clear, but it required more class definitions and setup than the Lisp version.

Java class-style flow



Functional Implementation — Common Lisp

Focus: recursive list processing and functional evaluation.

Language / Folder

Common Lisp / functional
Runs through SBCL and run_all.sh.

How input is represented

MiniScheme expressions are naturally list-shaped, so the Lisp version can read and process expressions in a structure that already looks close to Scheme.

How evaluation works

The evaluator works recursively. It checks the expression form, evaluates subexpressions, uses the environment for bindings, and creates closures for lambdas.

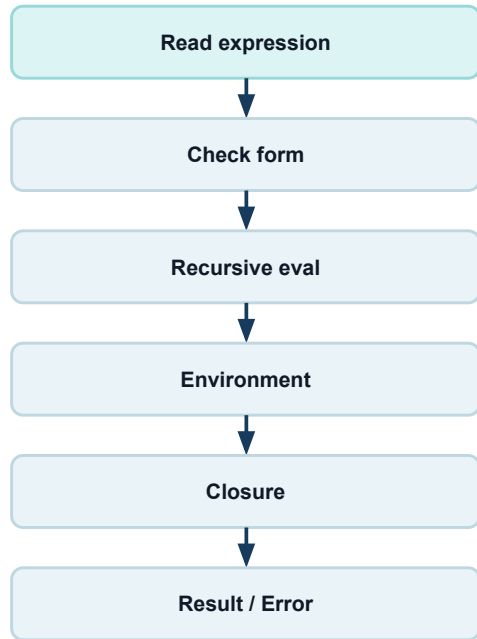
Main strength

The main strength is that Lisp fits a Scheme-like interpreter naturally. Recursive evaluation and list processing match the problem well.

Main challenge

The tradeoff is readability at first. Lisp syntax was less familiar, especially parentheses, assoc-style environments, and closure behavior.

Lisp evaluation flow



'cond' Add-On

'cond' works like a cleaner chain of if statements: check clauses in order and return the first true result.

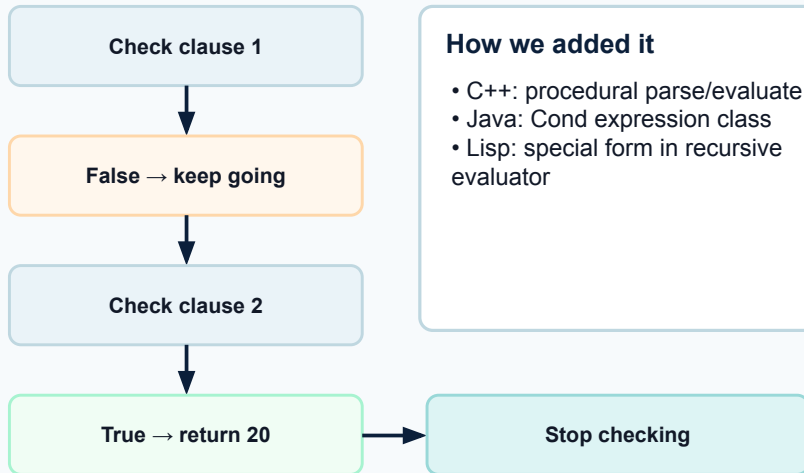
```
(cond
  (< 3 2) 10)
(= 3 3) 20)
(else 30))
```

Example explanation

Clause 1: ($< 3\ 2$) is false, so the evaluator keeps going.

Clause 2: ($= 3\ 3$) is true, so the evaluator returns 20 and stops.

The else branch is only used if none of the earlier conditions are true.



How we added it

- C++: procedural parse/evaluate logic
- Java: Cond expression class
- Lisp: special form in recursive evaluator

Key point

All three versions had to support ordered clause evaluation and correct else handling, even though each paradigm represented it differently.

Comparative Analysis

Same MiniScheme behavior, different design styles depending on the paradigm.

Category	C++ Procedural	Java OOP	Common Lisp Functional
Representation	Nodes / structures	Expression classes	Lists / symbolic expressions
Organization	Direct step-by-step logic	Separated responsibilities	Recursive functions
Environment	Explicit environment structure	Environment object	Association-style bindings
Strength	Direct control	Clean class structure	Natural fit for Scheme
Challenge	Manual handling	More boilerplate	Unfamiliar Lisp syntax

Most direct control

C++ made the evaluation steps very explicit, but required more manual structure and error handling.

Most organized structure

Java made responsibilities easier to separate through expression classes, environments, and closures.

Most natural fit

Common Lisp fit the interpreter logic well because MiniScheme expressions are already list-like.